

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1110

COURSE OUTCOME COVERED

***CO1:** Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)*

Assessment Tool Weightage: 30%

QUESTIONS: 6

Total Marks:30

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I Tejaswini.M(192411079) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Tejaswini.M

Annexure A
Short Answer Test

1. Suppose a software system is developed for a **Smart Banking System** where customers can open accounts, transfer funds, and pay utility bills.

(a) Explain the following object-oriented concepts:

Class

A class is a blueprint that defines the attributes and methods of an object.

Example: BankAccount

Object

An object is an instance of a class.

Example: Customer's Savings Account.

Message Passing

Objects communicate by sending messages (calling methods).

Example: Account.transferFunds().

Encapsulation

Encapsulation hides data and allows access only through methods.

Example: Balance is private and updated through deposit() and withdraw().

Polymorphism

One interface with multiple implementations.

Example: Different account types calculate interest differently.

(b) Describe the following object relationships:

Association

A general relationship between two classes.

Example: Customer ↔ BankAccount.

Aggregation

A weak "has-a" relationship.

Example: Bank has Employees.

Composition

A strong "part-of" relationship.

Example: BankAccount contains Transactions.

(c) Explain how these concepts improve software reusability and maintainability in a banking system.

- Reusable classes reduce code duplication.
 - Encapsulation protects data.
 - Polymorphism simplifies extension.
 - Relationships organize the system.
 - Easy maintenance and debugging.
-

2. Consider developing a **Smart Healthcare Management System**.

(a) Explain the following object-oriented design principles:

Abstraction

Shows only essential features while hiding implementation.

Encapsulation

Protects data by restricting direct access.

Modularity

Divides software into independent modules.

(b) Describe the following SOLID principles:

Liskov Substitution Principle (LSP)

A child class should replace its parent without changing program behavior.

Interface Segregation Principle (ISP)

Clients should not depend on methods they don't use.

(c) Explain how these principles improve software quality and flexibility.

- Improves flexibility
- Easy maintenance
- Better code reuse

- Reduces complexity
 - Improves software quality
-

3. Suppose a software company is developing an **Online Food Delivery System**.

(a) Explain the phases of the Software Development Life Cycle (SDLC):

1. Planning

Planning is the first phase of SDLC. In this phase, the project goals, scope, budget, schedule, and feasibility are determined. The development team identifies the resources required and prepares a project plan.

Purpose: To define the project's objectives and determine whether the project is feasible.

2. Requirement Analysis

In this phase, the requirements are collected from customers and stakeholders. Functional and non-functional requirements are analyzed and documented in a Software Requirement Specification (SRS).

Purpose: To understand what the customer expects from the software.

3. Design

The design phase converts the requirements into a blueprint for software development. It includes system architecture, database design, user interface design, and module design.

Purpose: To create the overall structure and architecture of the software.

4. Development

During this phase, programmers write the source code based on the design specifications. Each module is developed and integrated with other modules.

Purpose: To build the software according to the design.

5. Testing

The developed software is tested to identify and fix bugs. Different types of testing, such as unit testing, integration testing, system testing, and user acceptance testing, are performed to ensure the software meets the specified requirements.

Purpose: To verify that the software is free from defects and works correctly.

6. Maintenance

After deployment, the software is monitored and maintained. Bugs are fixed, performance is improved, and new features or updates are added based on user feedback.

Purpose: To keep the software reliable, secure, and up to date after release.

(b) Describe the purpose of each SDLC phase.

Planning

Defines project objectives.

Requirement Analysis

Collects customer requirements.

Design

Creates system architecture.

Development

Implements the software.

Testing

Finds and fixes defects.

Maintenance

Updates and improves the software.

(c) Explain how SDLC contributes to delivering reliable software.

- Produces reliable software
- Reduces development risk
- Improves quality
- Controls cost and time
- Ensures customer satisfaction

4. Consider a project for developing a **Smart Agriculture Monitoring System**, where customer requirements change regularly.

(a) Explain the following SDLC models:

Prototype Model

Builds a prototype before the final product.

Spiral Model

Focuses on risk analysis through iterative cycles.

Agile Model

Develops software in short iterations with continuous customer feedback.

DevOps Model

Combines development and operations using automation and continuous delivery.

(b) Compare the Prototype Model and Agile Model.

Prototype Model	Agile Model
Builds prototype first	Delivers working software in iterations
Requirements initially unclear	Requirements can change anytime
Customer validates prototype	Customer involved throughout
Suitable for unclear requirements	Suitable for changing requirements

(c) Recommend the most suitable SDLC model for this project with justification.

Agile Model

Reason:

- Handles changing requirements
- Frequent customer feedback
- Faster delivery
- Flexible development

5. Suppose a software analyst is designing a **Smart Transport Management System**.

(a) Explain the importance of UML in object-oriented software development.

- Visualizes software design
- Improves communication
- Simplifies system understanding
- Detects design errors early
- Provides documentation

(b) Describe the following UML diagrams:

State Diagram

Shows object states and transitions.

Component Diagram

Shows software components and dependencies.

Deployment Diagram

Shows hardware nodes and software deployment.

Package Diagram

Groups related classes into packages.

(c) Explain how UML models improve communication among stakeholders.

- Better communication among stakeholders
 - Eases development
 - Improves documentation
 - Simplifies maintenance
 - Reduces misunderstandings
-

6. Consider developing a **Smart Inventory Management System**.

(a) Explain the concept of modular software design.

Modular software design is a software development approach in which a large software system is divided into small, independent modules. Each module performs a specific function and can be developed, tested, maintained, and reused independently.

(b) Compare the procedural programming approach with the object-oriented programming approach.

Procedural Programming	Object-Oriented Programming
Function-based	Object-based
Data and functions separate	Data and methods together
Less secure	More secure (Encapsulation)
Less reusable	Highly reusable

(c) Explain how modularity improves software scalability, maintainability, and reusability.

- Improves scalability
- Easy maintenance
- High code reusability
- Simplifies testing
- Reduces development time

RUBRICS FOR CALCULATION

SHORT ANSWER TEST

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	20%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps